

Ciencias de la Computación VIII

Ing. Axel Benavides



Proyecto Ethernet

Miguel Rodríguez -20004742

Sandra Soria – 20002619

Gabriela Navarro - 20000127

BeagleBone Black pruebas Ethernet

Implementación de Ethernet en la BeagleBone Black con nuttx

La primera forma de implementación fue hacerlo de forma nativa, utilizando la interfaz de Ethernet LAN8710A. La idea era que, a través de esta interfaz se pudiera obtener conexión IP y comprobar utilizando ping.

Configuración menuconfig

- Networking Support -> Enable /
- Networking Support -> ICMP networking support -> IPPROTO_ICMP socket support
- Link layer support -> Late Driver /
- TCP/IP Networking -> Enable /
- Collect network statistics -> Enable -> Network device driver statistics
- Socket Support -> Socket options /
- RTOS Features -> Work queue -> Enable /
- network utilities -> network initialization -> hardware no mac address /
- network device/phy support -> network Device driver statistics y upper-half mdio
- network initialization -> network initialization thread
- Network Utilities → Network initialization → IP Address Configuration -> use dhcp to get ip address
- activar config_arm335x_cpsw (es emac para la beagle) Ethernet Subsystem
- Application Configuration -> Network Utilities -> ICMP Ping /
- Application Configuration -> System Libraries and NSH -> Ping /
- Device Drivers -> CLK /
- File Systems -> PROCFS File System
- Application -> Examples -> Network test
- System Type -> TI AM 335X Peripheral Support - Ethernet Subsystem

Al terminar de hacer la configuración y ejecutar la compilación, se obtuvo el siguiente error de arm_netinitialize, ya que el driver de Ethernet no fue implementado para la BeagleBone Black. Para evitar este problema, en el menuconfig se habilitó la opción de networking support-> Link layer support -> Late driver initialization. De esta forma, no se inicializa el driver de red cuando arranca y se puede obtener la compilación sin errores.

```
Register: ping
CC: chip/am335x_clockconfig.c chip/am335x_clockconfig.c: In function 'am335x_clockconfig':
chip/am335x_clockconfig.c:615:4: warning: #warning Missing logic [-Wcpp]
  615 | # warning Missing logic
      | ~~~~~
CC: chip/am335x_lowputc.c chip/am335x_lowputc.c: In function 'am335x_lowsetup':
chip/am335x_lowputc.c:191:2: warning: #warning Missing logic [-Wcpp]
  191 | #warning Missing logic
      | ~~~~~
CC: chip/am335x_serial.c chip/am335x_serial.c: In function 'am335x_uart0config':
chip/am335x_serial.c:571:2: warning: #warning Missing logic [-Wcpp]
  571 | #warning Missing logic
      | ~~~~~
chip/am335x_serial.c:574:2: warning: #warning Missing logic [-Wcpp]
  574 | #warning Missing logic
      | ~~~~~
CPP: /home/sandra-soria/nuttx-bbb-project/nuttx/boards/arm/am335x/beaglebone-black/scripts/sdram.ld-> /ho
LD: nuttx
arm-none-eabi-ld: /home/sandra-soria/nuttx-bbb-project/nuttx/staging/libarch.a(arm_initialize.o): in funct
ion 'up_initialize':
arm_initialize.c:(.text.up_initialize+0xc): undefined reference to 'arm_netinitialize'
Memory region      Used Size  Region Size  %age Used
  isram:             0 B         63 KB         0.00%
  ocm0:              0 B         48 KB         0.00%
  ddr:            243452 B       352 MB         0.07%
make[1]: *** [Makefile:217: nuttx] Error 1
make: *** [tools/Unix.mk:552: nuttx] Error 2
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H: ~/nuttx-bbb-project/nuttx$
```

```
.config - NuttX/x86_64 Configuration
-> Networking Support -> Link layer support
                                Late driver initialization

CONFIG_NETDEV_LATEINIT:

Normally, networking initialization occur in the later phase of the
boot process in the function up_initialize() when it calls the
driver initialization function, xxx_netinitialize(). This
initialization occurs after a sufficient amount of the OS has been
initialized so that driver registration can be performed, but
before the completion of OS initialization and before the first
application is started.

In a few situations, however, you may want to suppress this early
network driver initialization. As examples:

- If you are using SLIP or PPPD, then there will be no network
  driver to be initialized,
- Certain multi-network configurations where a simple call to
  xxx_netinitialize() may be insufficient, and
- Situations where there are other board-level hardware
  dependencies so that the hardware is not in an appropriate
  state for xxx_netinitialize() to be called.

( 59%)
< Exit >
```

Después de solucionar el error anterior toda la configuración funcionó correctamente, ya que no obtuvimos ningún error en la compilación, sin embargo, al ejecutar el ifconfig notamos que no aparece la interfaz eth0, el cual corresponde al Ethernet. Por lo que no reconoce el eth0, aunque se realice la configuración.

```
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H: ~/nuttx-bbb project/nuttx
cat      exit      ps      true      wait

Builtin Apps:
dd        ping      sh        udp_server
nsh       raw_udp_client  udp_client

nsh> ifconfig
lo        Link encap:Local Loopback at RUNNING mtu 1518
inet addr:127.0.0.1 DRaddr:127.0.0.1 Mask:255.0.0.0

RX: Received Fragment Errors Bytes
00000000 00000000 00000000 0
IPv4 ARP Dropped
00000000 00000000 00000000
TX: Queued Sent Errors Timeouts Bytes
00000000 00000000 00000000 00000000 0
Total Errors: 00000000

Received IPv4 TCP UDP ICMP
0000 0000 0000 0000
Dropped 0000 0000 0000 0000
IPv4 VHL: 0000 Frg: 0000
Checksum 0000 0000 0000 ----
TCP ACK: 0000 SYN: 0000
RST: 0000 0000
Type 0000 ---- ---- 0000
Sent 0000 0000 0000 0000
Rexmit ---- 0000 ---- ----

nsh>
```

Como no logramos que funcionara la conexión por Ethernet, continuamos con la implementación, pero de forma local, a través de loopback. En este caso pudimos observar que la configuración si se aplicó de forma correcta, porque al realizar un ping a la dirección 127.0.0.1 si obteníamos respuesta. Por lo que pudimos verificar el funcionamiento del laboratorio 1.

```
nsh> ping 127.0.0.1
PING 127.0.0.1 56 bytes of data
56 bytes from 127.0.0.1: icmp_seq=0 time=0.0 ms
56 bytes from 127.0.0.1: icmp_seq=1 time=0.0 ms
56 bytes from 127.0.0.1: icmp_seq=2 time=0.0 ms
56 bytes from 127.0.0.1: icmp_seq=3 time=0.0 ms
56 bytes from 127.0.0.1: icmp_seq=4 time=0.0 ms
56 bytes from 127.0.0.1: icmp_seq=5 time=0.0 ms
56 bytes from 127.0.0.1: icmp_seq=6 time=0.0 ms
56 bytes from 127.0.0.1: icmp_seq=7 time=0.0 ms
56 bytes from 127.0.0.1: icmp_seq=8 time=0.0 ms
56 bytes from 127.0.0.1: icmp_seq=9 time=0.0 ms
10 packets transmitted, 10 received, 0% packet loss, time 10100 ms
rtt min/avg/max/mdev = 0.000/0.000/0.000/0.000 ms
nsh>
```


En la siguiente imagen podemos observar el funcionamiento del laboratorio 1 utilizando loopback.

```
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H: ~
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H: ~/nuttx-bbb-project/nuttx
56 bytes from 127.0.0.1: icmp_seq=9 time=0.0 ms
10 packets transmitted, 10 received, 0% packet loss, time 10100 ms
rtt min/avg/max/mdev = 0.000/0.000/0.000/0.000 ms
nsh> udp_server &
udp_server [6:100]
nsh> === Servidor UDP (Loopback) ===
Iniciando en 127.0.0.1:5030
Servidor listo. Esperando conexiones...

ps
  PID  PPID  GROUP  PRI  POLICY  TYPE  NPX  STATE  EVENT  SIGMASK  STACK  COMMAND
    0     0     0    0  FIFO   Kthread  -  Ready          0000000000000000 0001008 Idle_Task
    1     0     0  224  RR     Kthread  -  Waiting Semaphore 0000000000000000 0001984 hpwork 0x8a0370dc 0
x8a037124
    2     0     0  100  RR     Kthread  -  Waiting Semaphore 0000000000000000 0001984 lpwork 0x8a037078 0
x8a0370c0
    3     0     3  100  RR     Task     -  Running          0000000000000000 0002008 nsh_main
    6     0     6  100  RR     Task     -  Waiting Semaphore 0000000000000000 0002008 udp_server
nsh> udp_client
Conectado al servidor UDP: 127.0.0.1:5030
Ingrese la operación a realizar: 2*5
< 127.0.0.1 client [1970/01/01 00:18:56] UDP: 2*5
> 127.0.0.1 client [1970/01/01 00:18:56] UDP: 2*5
< 127.0.0.1 server [1970/01/01 00:18:56] UDP: 10
Procesado: 2*5 -> 10
> 127.0.0.1 server [1970/01/01 00:18:56] UDP: 10
Ingrese la operación a realizar: 
```

¿Cómo utilizar Raw Sockets en NuttX?

Como contábamos con el laboratorio 7 de Raw Sockets para TCP en C, se utilizó esta base para modificar los encabezados para UDP y hacer pruebas de esta manera. Una vez con este código funcionando en Linux se exportó a NuttX y ocurrieron errores en compilación, esto es debido a que las librerías que se utilizan para Raw Sockets son completamente diferentes y muchos comandos no tienen soporte en NuttX, por lo cual, se buscó una ruta alternativa.

El primer paso fue entrar a la documentación de NuttX y ver qué ejemplo daban:

<https://nuttx.apache.org/docs/12.5.1/components/net/pkt.html>

Tras ver el ejemplo de la página quedó una interrogante, en el método send se envía un buffer, pero ¿cómo se arma este buffer? ya que no se da un ejemplo en la página. Para ello buscamos en repositorios de Github usos de la línea:

```
nt sd = socket(AF_PACKET, SOCK_RAW, 0);
```

Y con ello encontramos varios repositorios, el que utilizamos como referencia fue el siguiente:

https://github.com/NemoDacremont/linux/blob/e6d351ac62d3510b59ec2e1d81adf45958b298f2/pfa/src/send_udp.c

Donde vimos que en el buffer se envía el frame cómo tal en un arreglo de bytes, donde primero va la capa Ethernet, luego el header IP, el header UDP y finalmente el payload.

Entonces tuvimos que armar nuestro propio payload modificando cada una de las capas, el primero que se utilizó para probar fue el siguiente:

```
// Frame Ethernet + IPv4 + UDP
// IP src: 192.168.0.50
// IP dst: 192.168.0.18
// UDP src port: 4000
// UDP dst port: 5030
// payload: "2+2"

unsigned char bytes[] = {
    // ETH
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, // MAC destino (broadcast)
    0x00, 0xe0, 0xde, 0xad, 0xbe, 0xef, // MAC origen
    0x08, 0x00, // EtherType = IPv4

    // IP
    0x45, 0x00, // Version/IHL, TOS
    0x00, 0x1f, // Total Length = 31 (20 IP + 8 UDP + 3 datos)
    0x00, 0x01, // Identification
    0x40, 0x00, // Flags/Fragment
    0x40, // TTL = 64
    0x11, // Protocolo = UDP (17)
    0xb9, 0x38, // Checksum IP

    0xc0, 0xa8, 0x00, 0x32, // IP origen = 192.168.0.50
    0xc0, 0xa8, 0x00, 0x12, // IP destino = 192.168.0.18

    // UDP
    0x0f, 0xa0, // Puerto origen = 4000
    0x13, 0xa6, // Puerto destino = 5030
    0x00, 0x0b, // Longitud UDP = 11 (8 + 3)
    0xf6, 0xd1, // Checksum UDP

    // Payload "2+2"
    0x32, 0x2b, 0x32 // '2' '+' '2'
};
```


Raw Socket en BBB – Alternativa 1

Se trató de crear el socket utilizando AF_PACKET, pero este mostró un error y el sistema se detuvo. Creemos que esto ocurre porque AF_PACKET trabaja con encabezados de capa de enlace, mientras que loopback solo maneja comunicación a nivel de capa de transporte ya que es la misma IP, pero distintos puertos.

```
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H: ~  
hello ping sh udp_server  
nsh> raw_udp_client  
deadbeefcafedeadbeefcafe080045000028efc340004011d1abc0a8fc01c0a8fc02a076270f00142ce97476616c76327c73656e640a  
arm_prefetchabort: Prefetch abort. PC: 00000000 IFAR: 00000000 IFSR: 00000008  
dump_assert_info: Current Version: NuttX 12.11.0 11e81d1a89-dirty Nov 20 2025 13:51:31 arm  
dump_assert_info: Assertion failed panic: at file: :0 task: lpwork process: Kernel 0x8a002f64  
up_dump_register: R0: 8a038fd8 R1: 8a03c268 R2: 8a037f38 R3: 00000036  
up_dump_register: R4: 8a03a630 R5: 8a038fd8 R6: 00000000 R7: 00000000  
up_dump_register: R8: 8a038fc0 SB: 000005e4 SL: 6000001f FP: 8a036c6c  
up_dump_register: IP: 00000000 SP: 8a03cab0 LR: 8a014f84 PC: 00000000  
up_dump_register: CPSR: 2000001f  
dump_stackinfo: User Stack:  
dump_stackinfo: base: 0x8a03c378  
dump_stackinfo: size: 00001984  
dump_stackinfo: sp: 0x8a03cab0  
stack_dump: 0x8a03ca90: 00000000 8a038fc0 000005e4 6000001f 8a036c6c 00000000 00000000 2000001f  
stack_dump: 0x8a03cab0: 8a038fd8 8a036cac 00000000 00000000 8a038fc0 8a015030 8a038fd8 8a036cac  
stack_dump: 0x8a03cad0: 8a005818 00000000 8a038fc0 000005e4 6000001f 8a005840 8a036c64 8a003070  
stack_dump: 0x8a03caf0: 00000000 8a036c74 8a036cb4 00000000 00000000 8a03c268 00000002 00000000  
stack_dump: 0x8a03cb10: 00000000 00000000 00000000 00000000 00000000 8a003c74 00000000 00000000  
stack_dump: 0x8a03cb30: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000  
dump_tasks: PID GROUP PRI POLICY TYPE NPX STATE EVENT SIGMASK STACKBASE STACKSIZE COMMAND  
dump_tasks: ---- -  
dump_task: 0 0 0 FIFO Kthread - Ready 0000000000000000 0x8a03b2b0 1008 Idle_Tas  
k  
dump_task: 1 0 224 RR Kthread - Waiting Semaphore 0000000000000000 0x8a03baa0 1984 hpwork 0  
x8a036cc8 0x8a036d10  
dump_task: 2 0 100 RR Kthread - Running 0000000000000000 0x8a03c378 1984 lpwork 0  
x8a036c64 0x8a036cac  
dump_task: 3 3 100 RR Task - Waiting Semaphore 0000000000000000 0x8a03ced8 2008 nsh_main  
dump_task: 5 5 100 RR Task - Ready 0000000000000000 0x8a03e3e0 2008 raw_udp_  
client
```

Se probó crear un socket con la siguiente estructura: socket (AF_INET, SOCK_RAW, IPPROTO_UDP), con el objetivo de construir únicamente los encabezados IP y UDP para el caso de comunicación por loopback. Sin embargo, desplegaba este error indicando que no se podía crear el raw socket.

```
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H: ~/nuttx-bbb-project/nuttx$ make -j$(nproc)  
CC: sh_main.c raw_udp_client_main.c: In function 'raw_udp_client_main':  
raw_udp_client_main.c:38:37: error: 'IPPROTO_UDP' undeclared (first use in this function)  
38 | sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_UDP);  
|  
raw_udp_client_main.c:38:37: note: each undeclared identifier is reported only once for each function it appears in  
make[2]: *** [/home/sandra-soria/nuttx-bbb-project/apps/Application.mk:330: raw_udp_client_main.c.home.sandra-soria.n  
uttx-bbb-project.apps.examples.raw_udp_client.o] Error 1  
make[1]: *** [Makefile:54: /home/sandra-soria/nuttx-bbb-project/apps/examples/raw_udp_client_all] Error 2  
make[1]: *** Waiting for unfinished jobs....  
make: *** [tools/LibTargets.mk:248: /home/sandra-soria/nuttx-bbb-project/apps/libapps.a] Error 2
```


Con base a esto, se agregó el caso para SOCK_RAW con el objetivo de probar si se podía crear este tipo de socket. Sin embargo, al realizar la ejecución se obtuvo `errno = 97`, que indica `Address family not supported by protocol`.

```
sandre@beaglebone:~/nuttx-bsp$ ./nuttx-bsp-project/nuttx
BeagleBone: cape eeprom: i2c_probe: 0x57:
Net: eth0: MII MODE
cpsw, usb_ether
Press SPACE to abort autoboot in 0 seconds
=> load mmc 0 0x8a000000 nuttx.bin
226476 bytes read in 16 ms (13.5 MiB/s)
=> go 0x8a000000
## Starting application at 0x8A000000 ...

NuttsShell (NSH) NuttX-12.11.0
nsh> HELP
nsh: HELP: command not found
nsh> help
help usage: help [-v] [<cmd>]

.      cd      expr      ls      pwd      truncate
[      cp      false    mkdir   rm        name
?      cmp      fdinfo   mkfatfs rmdir     mount
alias  dirname   free     mkrd    set       unset
unalias df         help     mount   sleep     uptime
arp    dmesg     hexdump  mv       source    usleep
basename echo       ifconfig pidof    test      watch
break  exec      kill     printf  time      xd
cat    exit     pkill    ps       true      wait

Built-in Apps:
dd            ping          sh            udp_server
nsh           raw_udp_client  udp_client
```

Para solucionar el error 97, se modificó el archivo de NuttX `inet_sockif.c`, dentro de la función `inet_bind()`, en el cual se agregó el caso `SOCK_RAW` para que utilizara la misma lógica del `SOCK_DGRAM` y dejará de recibir “Protocol not supported”.

The image shows a code editor with two files open. The left file is `C:\inet_sock.c` and the right file is `C:\udp_client.c`. Both files have a `Makefile` and a `C` source file.

C:\inet_sock.c

```

net > inet > C:\inet_sock.c > @inet_bindsocket *, const sockaddr *, socklen_t
49 #ifdef HAVE_INET_SOCKETS
50 #if defined(NET_UDP_HAVE_STACK)
51     switch (psock->s_type)
52     {
53         case SOCK_DGRAM:
54             /* Bind a UDP/IP datagram socket */
55             ret = udp_bind(psock->s_conn, addr);
56         #else
57             /* Bind a UDP/TCP datagram socket */
58             ret = udp_bind(psock->s_conn, addr);
59         #endif
60     }
61     if (ret != -ENOSYS)
62     {
63         /* Bind a UDP/TCP datagram socket */
64         ret = udp_bind(psock->s_conn, addr);
65     }
66     if (ret != -ENOSYS)
67     {
68         /* Bind a UDP/TCP datagram socket */
69         ret = udp_bind(psock->s_conn, addr);
70     }
71     if (ret != -ENOSYS)
72     {
73         /* Bind a UDP/TCP datagram socket */
74         ret = udp_bind(psock->s_conn, addr);
75     }
76     if (ret != -ENOSYS)
77     {
78         /* Bind a UDP/TCP datagram socket */
79         ret = udp_bind(psock->s_conn, addr);
80     }
81     if (ret != -ENOSYS)
82     {
83         /* Bind a UDP/TCP datagram socket */
84         ret = udp_bind(psock->s_conn, addr);
85     }
86     if (ret != -ENOSYS)
87     {
88         /* Bind a UDP/TCP datagram socket */
89         ret = udp_bind(psock->s_conn, addr);
90     }
91     if (ret != -ENOSYS)
92     {
93         /* Bind a UDP/TCP datagram socket */
94         ret = udp_bind(psock->s_conn, addr);
95     }
96     if (ret != -ENOSYS)
97     {
98         /* Bind a UDP/TCP datagram socket */
99         ret = udp_bind(psock->s_conn, addr);
100    }
101    if (ret != -ENOSYS)
102    {
103        /* Bind a UDP/TCP datagram socket */
104        ret = udp_bind(psock->s_conn, addr);
105    }
106    if (ret != -ENOSYS)
107    {
108        /* Bind a UDP/TCP datagram socket */
109        ret = udp_bind(psock->s_conn, addr);
110    }
111    if (ret != -ENOSYS)
112    {
113        /* Bind a UDP/TCP datagram socket */
114        ret = udp_bind(psock->s_conn, addr);
115    }
116    if (ret != -ENOSYS)
117    {
118        /* Bind a UDP/TCP datagram socket */
119        ret = udp_bind(psock->s_conn, addr);
120    }
121    if (ret != -ENOSYS)
122    {
123        /* Bind a UDP/TCP datagram socket */
124        ret = udp_bind(psock->s_conn, addr);
125    }
126    if (ret != -ENOSYS)
127    {
128        /* Bind a UDP/TCP datagram socket */
129        ret = udp_bind(psock->s_conn, addr);
130    }
131    if (ret != -ENOSYS)
132    {
133        /* Bind a UDP/TCP datagram socket */
134        ret = udp_bind(psock->s_conn, addr);
135    }
136    if (ret != -ENOSYS)
137    {
138        /* Bind a UDP/TCP datagram socket */
139        ret = udp_bind(psock->s_conn, addr);
140    }
141    if (ret != -ENOSYS)
142    {
143        /* Bind a UDP/TCP datagram socket */
144        ret = udp_bind(psock->s_conn, addr);
145    }
146    if (ret != -ENOSYS)
147    {
148        /* Bind a UDP/TCP datagram socket */
149        ret = udp_bind(psock->s_conn, addr);
150    }
151    if (ret != -ENOSYS)
152    {
153        /* Bind a UDP/TCP datagram socket */
154        ret = udp_bind(psock->s_conn, addr);
155    }
156    if (ret != -ENOSYS)
157    {
158        /* Bind a UDP/TCP datagram socket */
159        ret = udp_bind(psock->s_conn, addr);
160    }
161    if (ret != -ENOSYS)
162    {
163        /* Bind a UDP/TCP datagram socket */
164        ret = udp_bind(psock->s_conn, addr);
165    }
166    if (ret != -ENOSYS)
167    {
168        /* Bind a UDP/TCP datagram socket */
169        ret = udp_bind(psock->s_conn, addr);
170    }
171    if (ret != -ENOSYS)
172    {
173        /* Bind a UDP/TCP datagram socket */
174        ret = udp_bind(psock->s_conn, addr);
175    }
176    if (ret != -ENOSYS)
177    {
178        /* Bind a UDP/TCP datagram socket */
179        ret = udp_bind(psock->s_conn, addr);
180    }
181    if (ret != -ENOSYS)
182    {
183        /* Bind a UDP/TCP datagram socket */
184        ret = udp_bind(psock->s_conn, addr);
185    }
186    if (ret != -ENOSYS)
187    {
188        /* Bind a UDP/TCP datagram socket */
189        ret = udp_bind(psock->s_conn, addr);
190    }
191    if (ret != -ENOSYS)
192    {
193        /* Bind a UDP/TCP datagram socket */
194        ret = udp_bind(psock->s_conn, addr);
195    }
196    if (ret != -ENOSYS)
197    {
198        /* Bind a UDP/TCP datagram socket */
199        ret = udp_bind(psock->s_conn, addr);
200    }
201    if (ret != -ENOSYS)
202    {
203        /* Bind a UDP/TCP datagram socket */
204        ret = udp_bind(psock->s_conn, addr);
205    }
206    if (ret != -ENOSYS)
207    {
208        /* Bind a UDP/TCP datagram socket */
209        ret = udp_bind(psock->s_conn, addr);
210    }
211    if (ret != -ENOSYS)
212    {
213        /* Bind a UDP/TCP datagram socket */
214        ret = udp_bind(psock->s_conn, addr);
215    }
216    if (ret != -ENOSYS)
217    {
218        /* Bind a UDP/TCP datagram socket */
219        ret = udp_bind(psock->s_conn, addr);
220    }
221    if (ret != -ENOSYS)
222    {
223        /* Bind a UDP/TCP datagram socket */
224        ret = udp_bind(psock->s_conn, addr);
225    }
226    if (ret != -ENOSYS)
227    {
228        /* Bind a UDP/TCP datagram socket */
229        ret = udp_bind(psock->s_conn, addr);
230    }
231    if (ret != -ENOSYS)
232    {
233        /* Bind a UDP/TCP datagram socket */
234        ret = udp_bind(psock->s_conn, addr);
235    }
236    if (ret != -ENOSYS)
237    {
238        /* Bind a UDP/TCP datagram socket */
239        ret = udp_bind(psock->s_conn, addr);
240    }
241    if (ret != -ENOSYS)
242    {
243        /* Bind a UDP/TCP datagram socket */
244        ret = udp_bind(psock->s_conn, addr);
245    }
246    if (ret != -ENOSYS)
247    {
248        /* Bind a UDP/TCP datagram socket */
249        ret = udp_bind(psock->s_conn, addr);
250    }
251    if (ret != -ENOSYS)
252    {
253        /* Bind a UDP/TCP datagram socket */
254        ret = udp_bind(psock->s_conn, addr);
255    }
256    if (ret != -ENOSYS)
257    {
258        /* Bind a UDP/TCP datagram socket */
259        ret = udp_bind(psock->s_conn, addr);
260    }
261    if (ret != -ENOSYS)
262    {
263        /* Bind a UDP/TCP datagram socket */
264        ret = udp_bind(psock->s_conn, addr);
265    }
266    if (ret != -ENOSYS)
267    {
268        /* Bind a UDP/TCP datagram socket */
269        ret = udp_bind(psock->s_conn, addr);
270    }
271    if (ret != -ENOSYS)
272    {
273        /* Bind a UDP/TCP datagram socket */
274        ret = udp_bind(psock->s_conn, addr);
275    }
276    if (ret != -ENOSYS)
277    {
278        /* Bind a UDP/TCP datagram socket */
279        ret = udp_bind(psock->s_conn, addr);
280    }
281    if (ret != -ENOSYS)
282    {
283        /* Bind a UDP/TCP datagram socket */
284        ret = udp_bind(psock->s_conn, addr);
285    }
286    if (ret != -ENOSYS)
287    {
288        /* Bind a UDP/TCP datagram socket */
289        ret = udp_bind(psock->s_conn, addr);
290    }
291    if (ret != -ENOSYS)
292    {
293        /* Bind a UDP/TCP datagram socket */
294        ret = udp_bind(psock->s_conn, addr);
295    }
296    if (ret != -ENOSYS)
297    {
298        /* Bind a UDP/TCP datagram socket */
299        ret = udp_bind(psock->s_conn, addr);
300    }
301    if (ret != -ENOSYS)
302    {
303        /* Bind a UDP/TCP datagram socket */
304        ret = udp_bind(psock->s_conn, addr);
305    }
306    if (ret != -ENOSYS)
307    {
308        /* Bind a UDP/TCP datagram socket */
309        ret = udp_bind(psock->s_conn, addr);
310    }
311    if (ret != -ENOSYS)
312    {
313        /* Bind a UDP/TCP datagram socket */
314        ret = udp_bind(psock->s_conn, addr);
315    }
316    if (ret != -ENOSYS)
317    {
318        /* Bind a UDP/TCP datagram socket */
319        ret = udp_bind(psock->s_conn, addr);
320    }
321    if (ret != -ENOSYS)
322    {
323        /* Bind a UDP/TCP datagram socket */
324        ret = udp_bind(psock->s_conn, addr);
325    }
326    if (ret != -ENOSYS)
327    {
328        /* Bind a UDP/TCP datagram socket */
329        ret = udp_bind(psock->s_conn, addr);
330    }
331    if (ret != -ENOSYS)
332    {
333        /* Bind a UDP/TCP datagram socket */
334        ret = udp_bind(psock->s_conn, addr);
335    }
336    if (ret != -ENOSYS)
337    {
338        /* Bind a UDP/TCP datagram socket */
339        ret = udp_bind(psock->s_conn, addr);
340    }
341    if (ret != -ENOSYS)
342    {
343        /* Bind a UDP/TCP datagram socket */
344        ret = udp_bind(psock->s_conn, addr);
345    }
346    if (ret != -ENOSYS)
347    {
348        /* Bind a UDP/TCP datagram socket */
349        ret = udp_bind(psock->s_conn, addr);
350    }
351    if (ret != -ENOSYS)
352    {
353        /* Bind a UDP/TCP datagram socket */
354        ret = udp_bind(psock->s_conn, addr);
355    }
356    if (ret != -ENOSYS)
357    {
358        /* Bind a UDP/TCP datagram socket */
359        ret = udp_bind(psock->s_conn, addr);
360    }
361    if (ret != -ENOSYS)
362    {
363        /* Bind a UDP/TCP datagram socket */
364        ret = udp_bind(psock->s_conn, addr);
365    }
366    if (ret != -ENOSYS)
367    {
368        /* Bind a UDP/TCP datagram socket */
369        ret = udp_bind(psock->s_conn, addr);
370    }
371    if (ret != -ENOSYS)
372    {
373        /* Bind a UDP/TCP datagram socket */
374        ret = udp_bind(psock->s_conn, addr);
375    }
376    if (ret != -ENOSYS)
377    {
378        /* Bind a UDP/TCP datagram socket */
379        ret =
```

Al tratar de ejecutar el `udp_server` obtuvimos un error en el `bind()`, en consecuencia, de los cambios realizados en `inet_sockif.c`. Además, al correr `raw_udp_client`, se logró crear el socket y que enviara, pero desplegó el error 89, el cual indica `Destination address required`.

```
404 switch (psock->s_type)
sandra-soria@sandra-seria-Legion-5-Pro-16ACH8E -
sandra-soria@sandra-seria-Legion-5-Pro-16ACH8E - /nuttx-bbb-project/nuttx

NuttShell (NSH) NuttX-12.11.0
nsh> help
help usage:  help [-v] [<cmd>]

.      cd      expr      ls      pwd      truncate
[      cp      false     mkdir   rm      uname
?      cmp      fdinfo    mkfatfs rmdir   umount
alias  dirname   free      mkrd    set     unset
unalias df        help      mount   sleep   uptime
arp    dmesg    hexdump   mv      source  usleep
basename echo      ifconfig  pidof   test    watch
break  exec      kill      printf  time    xd
cat    exit      pkill     ps      true    wait

Builtin Apps:
  dd      ping      sh      udp_server
  nsh      raw_udp_client  udp_client

nsh> udp_server &
udp_server [5:100]
nsh> === Servidor UDP (Loopback) ===
Iniciando en 127.0.0.1:5030
ERROR: Bind failed
```

```
404 switch (psock->s_type)
sandra-soria@sandra-seria-Legion-5-Pro-16ACH8E -
sandra-soria@sandra-seria-Legion-5-Pro-16ACH8E - /nuttx-bbb-project/nuttx

cat    exit    pkill   ps      true    wait

Builtin Apps:
  dd      ping      sh      udp_server
  nsh      raw_udp_client  udp_client

nsh> udp_server &
udp_server [5:100]
nsh> === Servidor UDP (Loopback) ===
Iniciando en 127.0.0.1:5030
ERROR: Bind failed
ps
  PID  PPID  GROUP  PRI  POLICY  TYPE  NPX  STATE  EVENT  SIGMASK  STACK  COMMAND
  0    0    0    0  FIFO   Kthread - Ready          0000000000000000 0001008 Idle_Task
  1    0    0  224  RR     Kthread - Waiting Semaphore 0000000000000000 0001984 hpwork 0x
8a036ff4 0x8a03703c
  2    0    0  100  RR     Kthread - Waiting Semaphore 0000000000000000 0001984 lpwork 0x
8a036f90 0x8a036fd8
  3    0    3  100  RR     Task   - Running          0000000000000000 0002008 nsh_main
nsh> raw_udp_client
Se creo el socket de manera correcta.
deadbeefcadedeadbeefc080045000028efc340004011d1abc0a8fc01c0a8fc02a076270f00142ce97476616c7632
7c73656e640a
send() returned -1, errno=89
nsh>
```


Al revisar nuevamente el archivo notamos que faltaba colocar un break en el caso del SOCK_DGRAM. Después de corregirlo, el servidor UDP se ejecutó correctamente, sin embargo, al correr udp_client apareció el error 88 indicando que el address ya está siendo utilizada.

```
49 #ifdef HAVE_INET_SOCKETS
66 #if defined(NET_UDP_HAVE_STACK)
450 FAR const struct sockaddr *addr, socklen_t addrlen)
484 switch (psock->s_type)
486 #ifdef CONFIG_NET_TCP
497 ret = -ENOSYS;
498 #endif
499 break;
500 #endif /* CONFIG_NET_TCP */
501 #endif
502 #ifdef CONFIG_NET_UDP
503 case SOCK_DGRAM:
504 {
505 #ifdef NET_UDP_HAVE_STACK
506 /* Bind a UDP/IP datagram socket */
507 ret = udp_bind(psock->s_conn, addr);
508 #else
509 nwarn("WARNING: UDP stack is not available in this "
510 "configuration\n");
511 ret = -ENOSYS;
512 #endif
513 #endif
514 break;
515 #endif
516 case SOCK_RAW:
517 #ifdef NET_UDP_HAVE_STACK
518 /* Bind a UDP/IP datagram socket */
519 ret = udp_bind(psock->s_conn, addr);
520 #else
521 nwarn("WARNING: UDP stack is not available in this "
522 "configuration\n");
523 ret = -ENOSYS;
524 #endif
525 #endif
526 break;
527 #endif /* CONFIG_NET_UDP */
528 #endif
529 #endif
530 #endif /* CONFIG_NET_UDP */
531 #endif
532 #endif
533 #if defined(CONFIG_NET_TCP) || defined(CONFIG_NET_UDP)
534 case SOCK_CTRL:
535 {
536 nerr("ERROR: Inappropriate socket type: %d\n", psock->s_type);
537 ret = -EOPNOTSUPP;
538 }
539 #endif
540 #endif
541 #endif
542 #endif
543 #endif
544 #endif
545 #endif
546 #endif
547 #endif
548 #endif
549 #endif
550 #endif
551 #endif
552 #endif
553 #endif
554 #endif
555 #endif
556 #endif
557 #endif
558 #endif
559 #endif
560 #endif
561 #endif
562 #endif
563 #endif
564 #endif
565 #endif
566 #endif
567 #endif
568 #endif
569 #endif
570 #endif
571 #endif
572 #endif
573 #endif
574 #endif
575 #endif
576 #endif
577 #endif
578 #endif
579 #endif
580 #endif
581 #endif
582 #endif
583 #endif
584 #endif
585 #endif
586 #endif
587 #endif
588 #endif
589 #endif
590 #endif
591 #endif
592 #endif
593 #endif
594 #endif
595 #endif
596 #endif
597 #endif
598 #endif
599 #endif
600 #endif
601 #endif
602 #endif
603 #endif
604 #endif
605 #endif
606 #endif
607 #endif
608 #endif
609 #endif
610 #endif
611 #endif
612 #endif
613 #endif
614 #endif
615 #endif
616 #endif
617 #endif
618 #endif
619 #endif
620 #endif
621 #endif
622 #endif
623 #endif
624 #endif
625 #endif
626 #endif
627 #endif
628 #endif
629 #endif
630 #endif
631 #endif
632 #endif
633 #endif
634 #endif
635 #endif
636 #endif
637 #endif
638 #endif
639 #endif
640 #endif
641 #endif
642 #endif
643 #endif
644 #endif
645 #endif
646 #endif
647 #endif
648 #endif
649 #endif
650 #endif
651 #endif
652 #endif
653 #endif
654 #endif
655 #endif
656 #endif
657 #endif
658 #endif
659 #endif
660 #endif
661 #endif
662 #endif
663 #endif
664 #endif
665 #endif
666 #endif
667 #endif
668 #endif
669 #endif
670 #endif
671 #endif
672 #endif
673 #endif
674 #endif
675 #endif
676 #endif
677 #endif
678 #endif
679 #endif
680 #endif
681 #endif
682 #endif
683 #endif
684 #endif
685 #endif
686 #endif
687 #endif
688 #endif
689 #endif
690 #endif
691 #endif
692 #endif
693 #endif
694 #endif
695 #endif
696 #endif
697 #endif
698 #endif
699 #endif
700 #endif
701 #endif
702 #endif
703 #endif
704 #endif
705 #endif
706 #endif
707 #endif
708 #endif
709 #endif
710 #endif
711 #endif
712 #endif
713 #endif
714 #endif
715 #endif
716 #endif
717 #endif
718 #endif
719 #endif
720 #endif
721 #endif
722 #endif
723 #endif
724 #endif
725 #endif
726 #endif
727 #endif
728 #endif
729 #endif
730 #endif
731 #endif
732 #endif
733 #endif
734 #endif
735 #endif
736 #endif
737 #endif
738 #endif
739 #endif
740 #endif
741 #endif
742 #endif
743 #endif
744 #endif
745 #endif
746 #endif
747 #endif
748 #endif
749 #endif
750 #endif
751 #endif
752 #endif
753 #endif
754 #endif
755 #endif
756 #endif
757 #endif
758 #endif
759 #endif
760 #endif
761 #endif
762 #endif
763 #endif
764 #endif
765 #endif
766 #endif
767 #endif
768 #endif
769 #endif
770 #endif
771 #endif
772 #endif
773 #endif
774 #endif
775 #endif
776 #endif
777 #endif
778 #endif
779 #endif
780 #endif
781 #endif
782 #endif
783 #endif
784 #endif
785 #endif
786 #endif
787 #endif
788 #endif
789 #endif
790 #endif
791 #endif
792 #endif
793 #endif
794 #endif
795 #endif
796 #endif
797 #endif
798 #endif
799 #endif
800 #endif
801 #endif
802 #endif
803 #endif
804 #endif
805 #endif
806 #endif
807 #endif
808 #endif
809 #endif
810 #endif
811 #endif
812 #endif
813 #endif
814 #endif
815 #endif
816 #endif
817 #endif
818 #endif
819 #endif
820 #endif
821 #endif
822 #endif
823 #endif
824 #endif
825 #endif
826 #endif
827 #endif
828 #endif
829 #endif
830 #endif
831 #endif
832 #endif
833 #endif
834 #endif
835 #endif
836 #endif
837 #endif
838 #endif
839 #endif
840 #endif
841 #endif
842 #endif
843 #endif
844 #endif
845 #endif
846 #endif
847 #endif
848 #endif
849 #endif
850 #endif
851 #endif
852 #endif
853 #endif
854 #endif
855 #endif
856 #endif
857 #endif
858 #endif
859 #endif
860 #endif
861 #endif
862 #endif
863 #endif
864 #endif
865 #endif
866 #endif
867 #endif
868 #endif
869 #endif
870 #endif
871 #endif
872 #endif
873 #endif
874 #endif
875 #endif
876 #endif
877 #endif
878 #endif
879 #endif
880 #endif
881 #endif
882 #endif
883 #endif
884 #endif
885 #endif
886 #endif
887 #endif
888 #endif
889 #endif
890 #endif
891 #endif
892 #endif
893 #endif
894 #endif
895 #endif
896 #endif
897 #endif
898 #endif
899 #endif
900 #endif
901 #endif
902 #endif
903 #endif
904 #endif
905 #endif
906 #endif
907 #endif
908 #endif
909 #endif
910 #endif
911 #endif
912 #endif
913 #endif
914 #endif
915 #endif
916 #endif
917 #endif
918 #endif
919 #endif
920 #endif
921 #endif
922 #endif
923 #endif
924 #endif
925 #endif
926 #endif
927 #endif
928 #endif
929 #endif
930 #endif
931 #endif
932 #endif
933 #endif
934 #endif
935 #endif
936 #endif
937 #endif
938 #endif
939 #endif
940 #endif
941 #endif
942 #endif
943 #endif
944 #endif
945 #endif
946 #endif
947 #endif
948 #endif
949 #endif
950 #endif
951 #endif
952 #endif
953 #endif
954 #endif
955 #endif
956 #endif
957 #endif
958 #endif
959 #endif
960 #endif
961 #endif
962 #endif
963 #endif
964 #endif
965 #endif
966 #endif
967 #endif
968 #endif
969 #endif
970 #endif
971 #endif
972 #endif
973 #endif
974 #endif
975 #endif
976 #endif
977 #endif
978 #endif
979 #endif
980 #endif
981 #endif
982 #endif
983 #endif
984 #endif
985 #endif
986 #endif
987 #endif
988 #endif
989 #endif
990 #endif
991 #endif
992 #endif
993 #endif
994 #endif
995 #endif
996 #endif
997 #endif
998 #endif
999 #endif
1000 #endif
```

```
221 #define ELIBSCN 81
/* Linux errno extension
39 return 1;

sandra.soria@sandra.soria-Legion-5-Pro-16ACH6H: ~
sandra.soria@sandra.soria-Legion-5-Pro-16ACH6H: ~
sandra.soria@sandra.soria-Legion-5-Pro-16ACH6H: ~

Builtin Apps:
dd ping sh udp_server
nsh raw_udp_client udp_client

nsh> udp_server &
udp_server [5:100]
nsh> === Servidor UDP (Loopback) ===
Iniciando en 127.0.0.1:5030
Servidor listo. Esperando conexiones...

ps
  PID PPID GROUP PRI POLICY TYPE NPX STATE EVENT SIGMASK STACK COMMAND
    0 0 0 0 FIFO Kthread - Ready 0000000000000000 0001008 Idle_Task
    1 0 0 224 RR Kthread - Waiting Semaphore 0000000000000000 0001984 hpwork 0x
8a03707c 0x8a0370c4
    2 0 0 100 RR Kthread - Waiting Semaphore 0000000000000000 0001984 lpwork 0x
8a037018 0x8a037060
    3 0 3 100 RR Task - Running 0000000000000000 0002008 nsh_main
    5 0 5 100 RR Task - Waiting Semaphore 0000000000000000 0002008 udp_serve

r
nsh> sleep 2
nsh> raw_udp_client
Se creo el socket de manera correcta.
bind() failed: errno=98
nsh>
```


Para comprender mejor como pasaban nuestros paquetes de NuttX, revisamos el archivo `inet_sendto`, el cual se encarga de manejar el envío para sockets de tipo UDP. Al probar el programa `raw_udp_client` en la BeagleBone Black, se pudo visualizar que el socket se creaba bien, sin embargo, al llamar a `send()` despliego el mensaje de `Inappropriate socket type 3`. Esto indica que el socket esta tratando de trabajar con sockets normales UDP.

```
C raw_udp_client_main.c  C inet_sockifc M X  C pkt_sendmsg.c  C socket.c  C socket.h  C in.h  C errno.h  C sendto.c M  C sendmsg.c M
net> inet> C inet_sockifc> @inet_sendto(socket *, const void *, size_t, int, const struct sockaddr *, socklen_t)
66
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
/* Verify that a valid address has been provided */
switch (to->sa_family)
{
#ifdef CONFIG_NET_IPV4
case AF_INET:
minlen = sizeof(struct sockaddr_in);
break;
#endif
#ifdef CONFIG_NET_IPV6
case AF_INET6:
minlen = sizeof(struct sockaddr_in6);
break;
#endif
default:
nerr("ERROR: Unrecognized address family: %d\n", to->sa_family);
return -EAFNOSUPPORT;
}
if (tolen < minlen)
{
nerr("ERROR: Invalid address length: %d < %d\n", tolen, minlen);
return -EADDRF;
}
#ifdef CONFIG_NET_UDP
if (psock->s_type != SOCK_DGRAM)
{
nerr("ERROR: Inappropriate socket type %d\n", psock->s_type);
return -EADDRF;
}
if (psock->s_type != SOCK_RAW)
{
nerr("ERROR: Inappropriate socket type %d\n", psock->s_type);
return -EADDRF;
}
}
/* Now handle the INET sendto() operation */
```

```
sendra-soria@sendra-soria-Legion-5-Pro-16ACH6H: ~
sendra-soria@sendra-soria-Legion-5-Pro-16ACH6H: ~$ cd /home/sendra-soria/projects/nuttx
sendra-soria@sendra-soria-Legion-5-Pro-16ACH6H: ~$ ls
.      cd      expr      ls        pwd        truncate
[      cp        false     mkdir     rm          uname
?      cmp        fdinfo    mknfs     rmdir      umount
alias  dirname   free      mkrd      set         unset
unalias df         help      mount     sleep      uptime
arp    dmesg     hexdump   mv         source     usleep
basename echo       ifconfig  pidof     test       watch
break  exec      kill      printf    time       xd
cat    exit      pkill     ps         true       wait

Builtin Apps:
dd      ping      sh        udp_server
nsh     raw_udp_client  udp_client

nsh> udp_server &
udp_server [5:100]
nsh> === Servidor UDP (Loopback) ===
Iniciando en el :5030
Servidor listo. Esperando conexiones...

raw_udp_client
Se creo el socket de manera correcta.
4500001f000140004011b9387f0000017f0000010fa013a6000bf6d1322b32
inet_sendto: ERROR: Inappropriate socket type 3
send() returned -1, errno=9
nsh>
```

Raw Socket BBB – Alternativa 2

Como una alternativa para solucionar las fallas se habilitaron las opciones debug para todo lo relacionado a red, con la finalidad de ver cual es la traza por la que pasa los raw_sockets antes que ocurra el crash.

Únicamente se corre esta línea antes del crasheo:

```
psock_send_eventhandler: flags: 0010 sent: 0
```

En vista que el error ocurre en ejecución del eventhandler nos dirigimos a net/pkt/pkt_sendmsg.c

Haciendo debug de esta función nos encontramos que el causante del crash es la siguiente línea:

```
dev->d_len = dev->d_sndlen;
```

Al comentar esa línea deja de ocurrir el crash.

En base a nuestro entendimiento lo que ocurre es que el driver de loopback a comparación del de ethernet no tiene un buffer de memoria para ingresar a la cola los paquetes y el crasheo ocurre en el work scheduler.

Ethernet-over-USB (RNDIS) en NuttX con la BBB.

Una alternativa para lograr la comunicación en red entre la computadora y la BeagleBone Black fue usar el protocolo RNDIS, el cual permite simular una conexión tipo ethernet a través del cable USB, sin tener la necesidad de utilizar un dispositivo externo. Para esto, nos apoyamos en la siguiente documentación, la cual nos sirvió como guía para realizar la configuración.

<https://nuttx.apache.org/docs/latest/guides/rndis.html>

<https://www.youtube.com/watch?v=8noH8v7xNgs&t=275s>

Configuración del menuconfig

- CONFIG_ARCH_ARM
- CONFIG_ARCH_ARMV7A
- CONFIG_ARCH_CORTEXA8
- CONFIG_ARCH_CHIP_AM335X
- CONFIG_ARCH_CHIP_AM3358
- CONFIG_ARCH_BOARD_BEAGLEBONE_BLACK
- CONFIG_ARCH_BOARD="beaglebone-black"
- CONFIG_AM335X_UART0
- CONFIG_AM335X_USB0

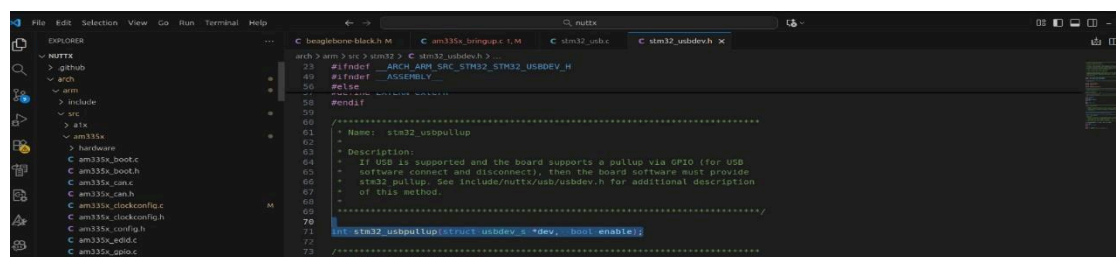
- CONFIG_USBDEV
- CONFIG_RNDIS
- CONFIG_RNDIS_VENDORID=0x584e
- CONFIG_RNDIS_VENDORSTR="NuttX"
- CONFIG_RNDIS_PRODUCTID=0x5342
- CONFIG_RNDIS_PRODUCTSTR="RNDIS gadget"
- CONFIG_RNDIS_SERIALSTR="1234"
- CONFIG_RNDIS_EPBULKIN=1
- CONFIG_RNDIS_EPBULKOUT=2
- CONFIG_RNDIS_EPINTIN=3
- CONFIG_NET
- CONFIG_NET_ETHERNET
- CONFIG_NET_LOOPBACK
- CONFIG_NET_IPv4
- CONFIG_NET_TCP
- CONFIG_NET_UDP
- CONFIG_NET_ICMP
- CONFIG_NET_ARP
- CONFIG_NET_PKT
- CONFIG_NET_WRITE_BUFFERS
- CONFIG_NET_READAHEAD
- CONFIG_NET_BINDTODEVICE
- CONFIG_NET_IPV4_CHECKSUMS
- CONFIG_NET_STATISTICS
- CONFIG_NETDEV_STATISTICS
- CONFIG_NETDEV_LATEINIT
- CONFIG_NETDEV_LPWORK_THREAD
- CONFIG_NETDEV_TELNET
- CONFIG_NETDEV_IFINDEX
- CONFIG_NETUTILS_NETINIT
- CONFIG_NETINIT_THREAD
- CONFIG_NETINIT_THREAD_STACKSIZE=2048
- CONFIG_NETINIT_THREAD_PRIORITY=80
- CONFIG_NETINIT_IPADDR=0x0a000002
- CONFIG_NETINIT_DRIPADDR=0x0a000001
- CONFIG_NETINIT_NETMASK=0xffffffff
- CONFIG_NETINIT_NOMAC
- CONFIG_NETINIT_SWMAC
- CONFIG_NETINIT_MACADDR_1=0xdeadbeef
- CONFIG_NETINIT_MACADDR_2=0x00e0

- CONFIG_NETUTILS_DHCPD
- CONFIG_NETUTILS_PING
- CONFIG_NETUTILS_TELNETD
- CONFIG_NSH_LIBRARY
- CONFIG_NSH_NETINIT
- CONFIG_NSH_CONSOLE
- CONFIG_NSH_TELNET
- CONFIG_SYSTEM_TELNETD
- CONFIG_SYSTEM_TELNETD_PROGNAME="telnetd"
- CONFIG_SYSTEM_TELNETD_PORT=23
- CONFIG_SYSTEM_TELNETD_PRIORITY=100
- CONFIG_SYSTEM_TELNETD_STACKSIZE=2048

Al ejecutar el comando make, se obtuvo este error y al revisar el código de NuttX identificamos que la función arm_usbinitialize no está implementada, por lo que es necesario desarrollarla para poder seguir probando con RNDIS.

```
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H: ~/nuttx-bbb-project/nuttx
191 | #warning Missing logic
    | ^~~~~~
CC: chip/am335x_serial.c chip/am335x_serial.c: In function 'am335x_uart0config':
chip/am335x_serial.c:571:2: warning: #warning Missing logic [-Wcpp]
  571 | #warning Missing logic
      | ^~~~~~
chip/am335x_serial.c:574:2: warning: #warning Missing logic [-Wcpp]
  574 | #warning Missing logic
      | ^~~~~~
CPP: /home/sandra-soria/nuttx-bbb-project/nuttx/boards/arm/am335x/beaglebone-black/scripts
/sdram.ld-> /home/sandra-soria/nuttx-bbb-project/nuttx/boards/arm/am335x/beaglebone-black/s
LD: nuttx
arm-none-eabi-ld: /home/sandra-soria/nuttx-bbb-project/nuttx/staging/libarch.a(arm_initiali
ze.o): in function 'up_initialize':
arm_initialize.c(.text.up_initialize+0xc): undefined reference to 'arm_usbinitialize'
Memory region      Used Size  Region Size  %age Used
isram:              0 B         63 KB         0.00%
ocmc0:              0 B         48 KB         0.00%
ddr:               245680 B      352 MB         0.07%
make[1]: *** [Makefile:217: nuttx] Error 1
make: *** [tools/Unix.mk:552: nuttx] Error 2
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H: ~/nuttx-bbb-project/nuttx$
```

Aunque implementemos la función arm_usbinitialize, no es posible realizar RNDIS en el AM335x, porque NuttX no tiene la implementación para el usbdev para la BeagleBone Black, a diferencia de otras tarjetas como la STM32F3Discovery, donde sí está implementada.



```

23 #ifndef _ARCH_ARM_SAC_STM32_STM32_USBDEV_H
24 #define _ARCH_ARM_SAC_STM32_STM32_USBDEV_H
49 #ifndef _ASSEMBLY_
50 #else
51 #endif
52
53 //*****
54 // Name: stm32_usbpullup
55 //
56 // Description:
57 // If USB is supported and the board supports a pullup via GPIO (for USB
58 // software connect and disconnect), then the board software must provide
59 // a stm32 pullup. See include/nuttx/usb/usbdev.h for additional description
60 // of this method.
61
62 //*****
63
64 #include <nuttx/usb/usbdev.h>
65
66 #define STM32_USB_PULLUP_ENABLE
67
68 #define STM32_USB_PULLUP_ENABLE
69
70 #define STM32_USB_PULLUP_ENABLE
71
72 #define STM32_USB_PULLUP_ENABLE
73
74 #define STM32_USB_PULLUP_ENABLE
75

```

Raspberry Pi Pico con Ethernet

Para esta implementación se utiliza la Raspberry Pi Pico como tarjeta principal y el módulo ENC28J60 como adaptador de Ethernet. Al combinar ambos dispositivos, la tarjeta que originalmente no tenía puerto de red adquiere la capacidad de poder comunicarse por Ethernet, lo que permite practicar el uso de sockets y poder enviar y recibir frames en la red.

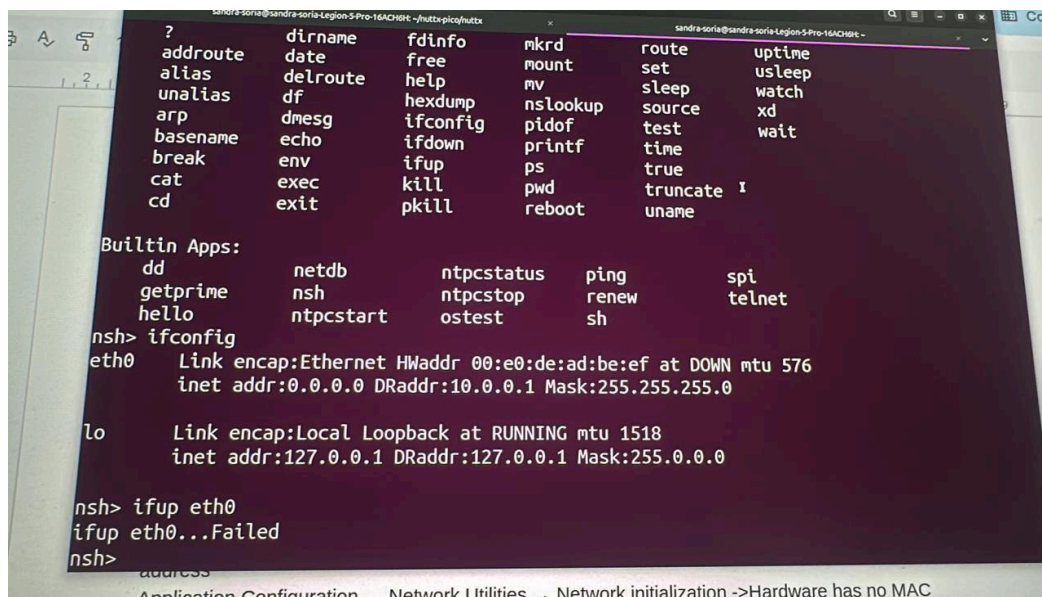
Pruebas de configuración de Raspberry Pi Pico:

Durante el proceso de conexión del módulo ENC28J60 a la Raspberry Pi Pico surgieron ciertos inconvenientes con el cableado. Uno de los problemas fue que al realizar la compilación se logró que no hubiera ningún error, sin embargo, al probar la parte del Ethernet la interfaz seguía apareciendo en estado de DOWN y, al intentar ejecutar el ifup nos devolvía failed.

Para guiarnos con la configuración utilizamos el siguiente video y documentación:

<https://www.youtube.com/watch?v=HrQpXO73GLU>

<https://qiita.com/yunkya2/items/02301a06f4ade334800e>



```
?      dirname  fdinfo    mkrd       route      uptime
addroute date      free      mount      set        usleep
alias   delroute  help      mv         sleep      watch
unalias df        hexdump   nslookup   source     xd
arp      dmesg     ifconfig  pidof      test       wait
basename echo      ifdown    printf     time
break    env       ifup      ps         true
cat       exec      kill      pwd        truncate
cd        exit      pkill     reboot     uname

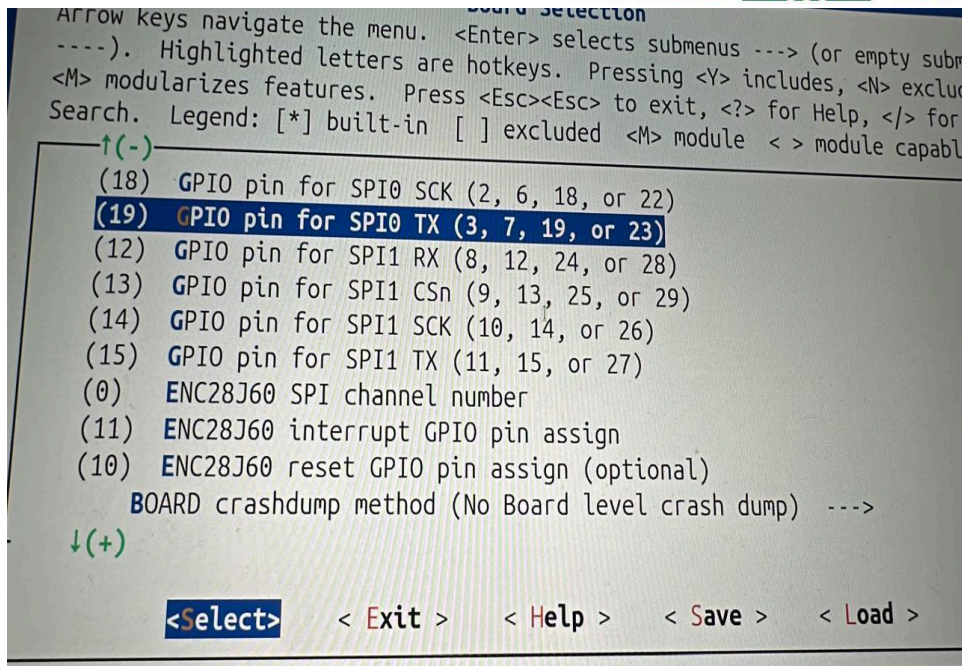
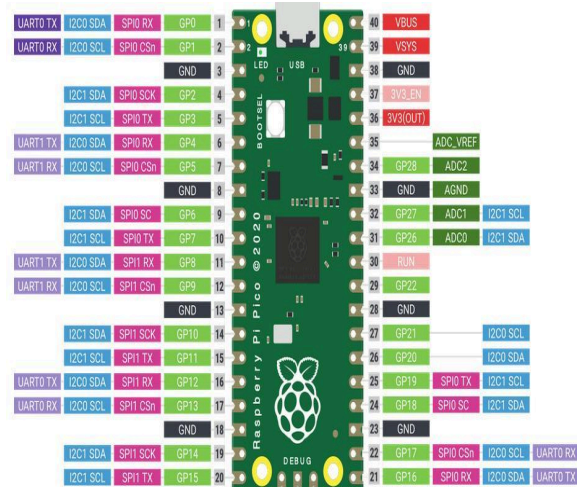
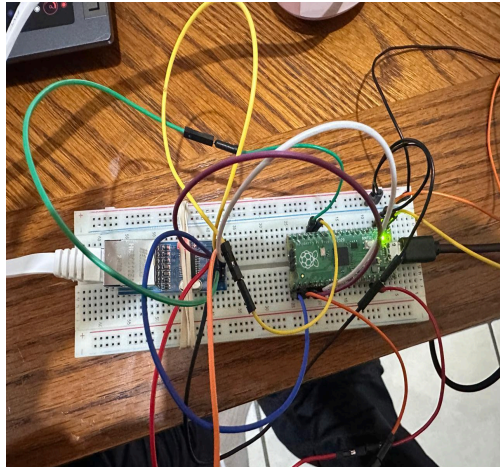
Builtin Apps:
dd        netdb     ntpcstatus ping       spi
getprime  nsh       ntpcstop  renew     telnet
hello     ntpcstart ostest    sh

nsh> ifconfig
eth0      Link encap:Ethernet HWaddr 00:e0:de:ad:be:ef at DOWN mtu 576
          inet addr:0.0.0.0 DRaddr:10.0.0.1 Mask:255.255.255.0

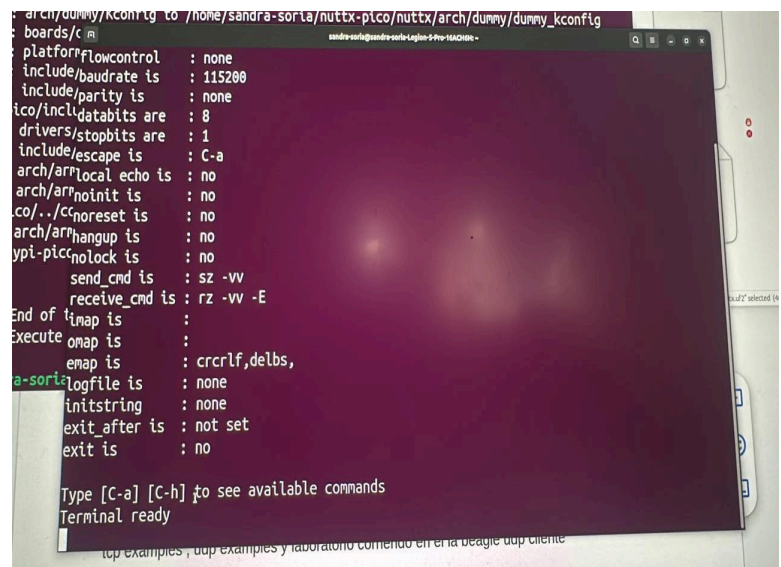
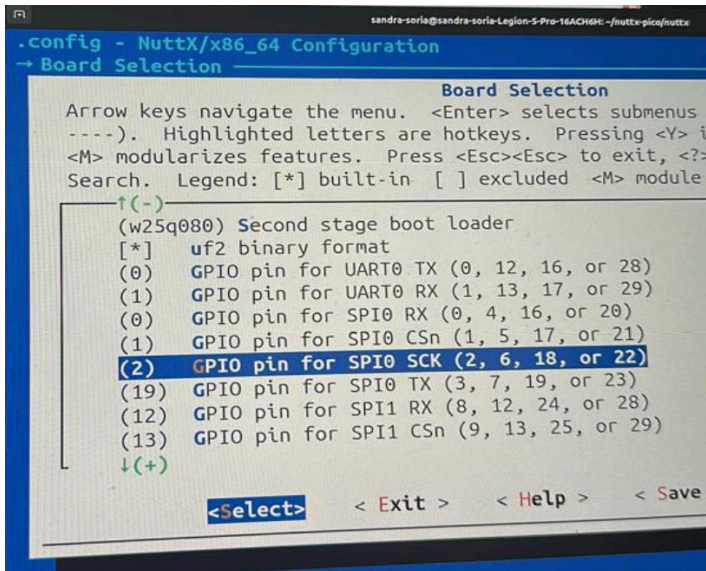
lo        Link encap:Local Loopback at RUNNING mtu 1518
          inet addr:127.0.0.1 DRaddr:127.0.0.1 Mask:255.0.0.0

nsh> ifup eth0
ifup eth0...Failed
nsh>
```

Por lo que al investigar nos dimos cuenta de que los pines del adaptador estaban mal conectados a la Pico, ya que al revisar la configuración del menuconfig únicamente se podía utilizar un SPI channel, pero nosotros estábamos utilizando las líneas de los dos SPI al mismo tiempo.



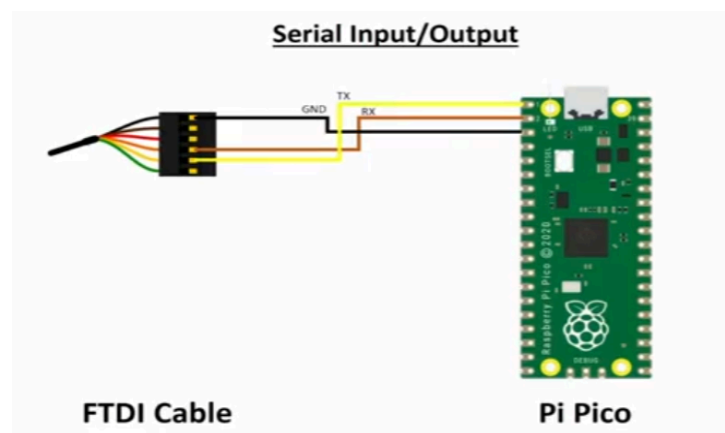
Al reubicar los pines se logró obtener correctamente la conexión del cableado, sin embargo, seguimos teniendo el problema de que no se mostraba nada en la consola. Por lo que, al investigar, nos dimos cuenta de que estamos utilizando los mismos GPIO tanto para el UART como para el SPI, por lo que no reconocía cuál debía utilizar, impidiendo el funcionamiento de la comunicación.



Debido a lo anterior, decidimos iniciar nuevamente desde cero trabajando únicamente con la parte de UART. Por lo que nos dimos cuenta de que la conexión entre el cable FTDI y la tarjeta estaba conectada incorrectamente, ya que habíamos conectado TX -> TX y RX -> RX. Esto está incorrecto ya que la línea TX es para enviar datos y RX es para recibir, de modo que uno envíe y otro reciba datos.

Para poder guiarnos y corregir el cableado, tomamos como referencia el siguiente video:

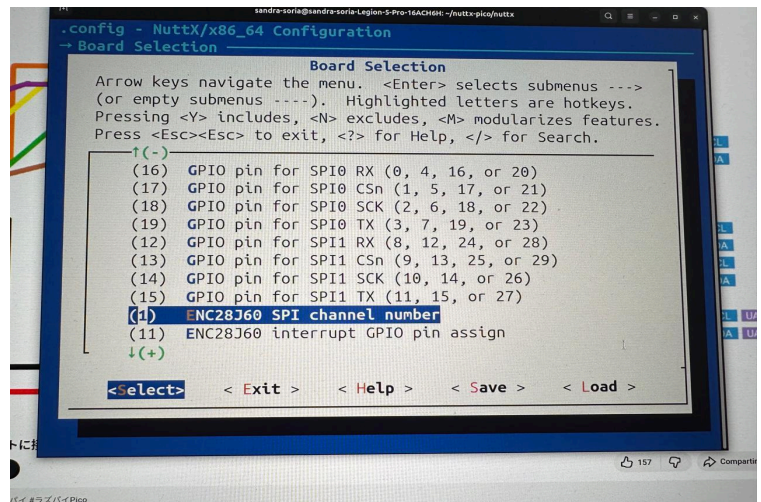
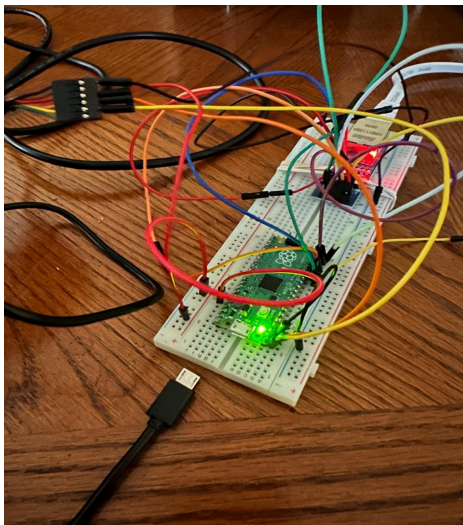
<https://www.youtube.com/watch?v=pbWhoJdYA1s&t=68s>



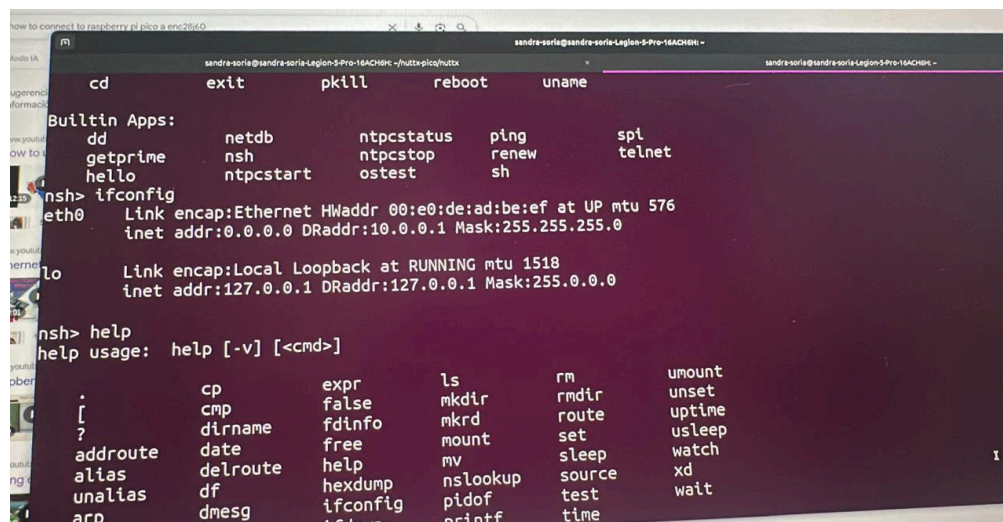
Entonces para solucionar este problema, decidimos guiarnos por la configuración estándar que ofrece el menuconfig, dejando el UART en los pines GPIO0 Y GPI1, y realizando la conexión del módulo a la tarjeta utilizando el SPI1.

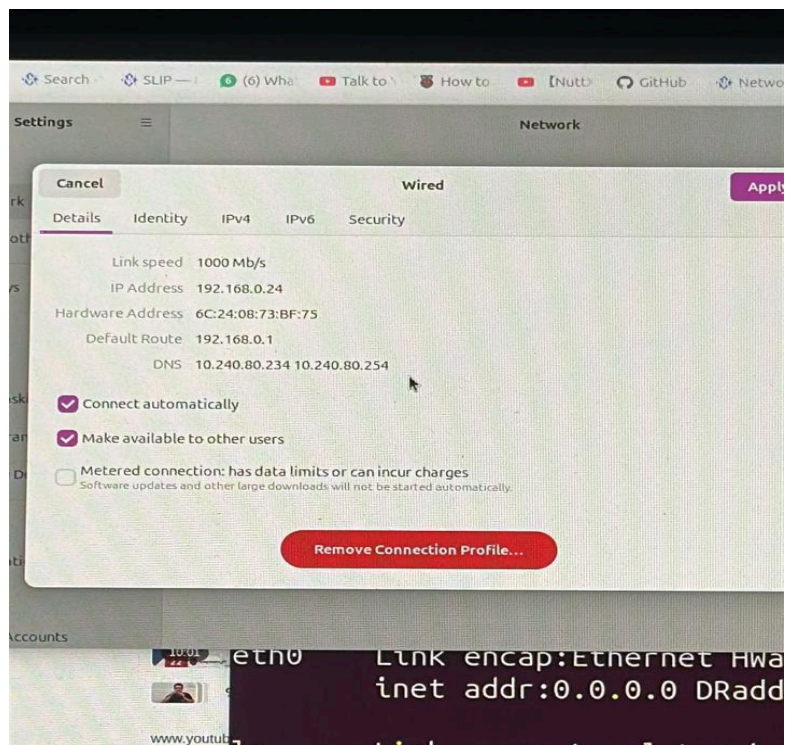
Para obtener la conexión correcta del cableado nos guiamos con la siguiente documentación:

<https://nuttx.apache.org/docs/latest/platforms/arm/rp2040/boards/raspberrypi-pico/index.html>

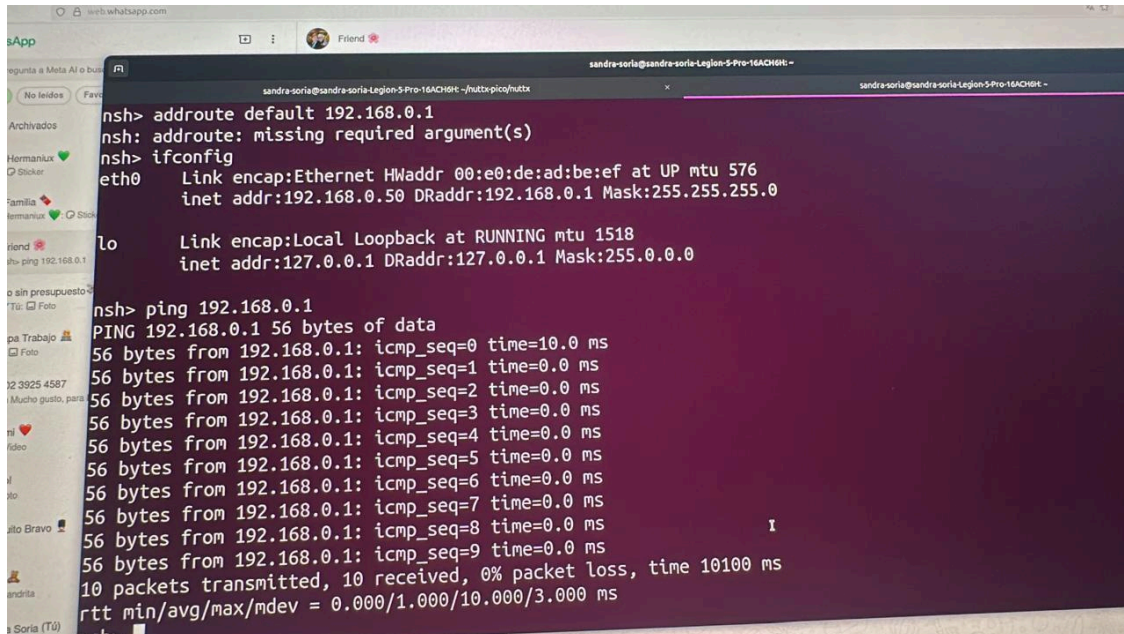


Otro problema que notamos fue que el cable USB que utilizamos para darle energía a la tarjeta Raspberry Pi Pico no proporcionaba la suficiente energía, por lo que los LEDs no encendían completamente. Debido a esto, decidimos utilizar el VCC del cable FTDI para darle energía al circuito. Al realizarlo se logró identificar de forma correcta el Ethernet.





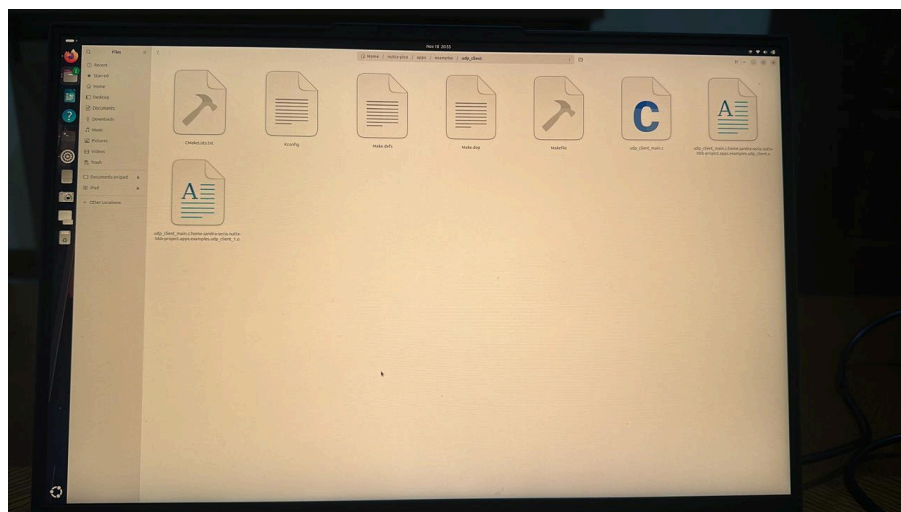
Una vez obtenida la dirección, se realizó el `addroute default`, el cual indicaba que faltaban argumentos, no sabíamos qué argumentos eran los necesarios por lo que en el `ifconfig` nos dimos cuenta de que automáticamente se colocaba la dirección y realizamos un `ping` para comprobar si había conexión, lo cual fue exitoso.



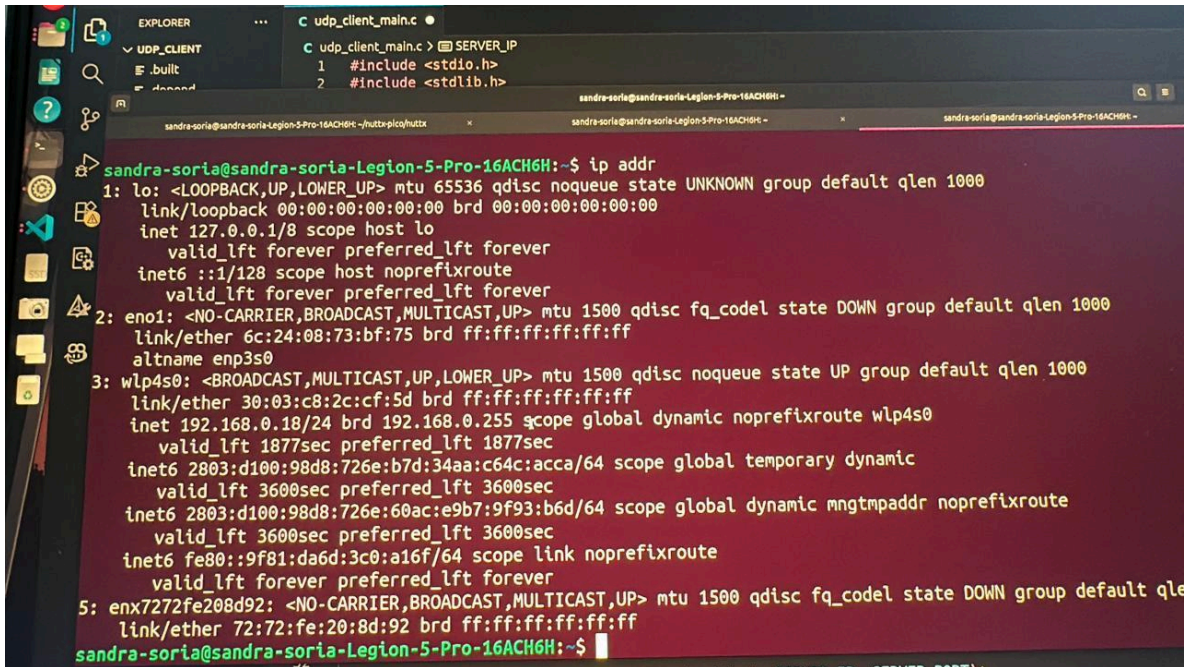
```
nsh> addroute default 192.168.0.1
nsh: addroute: missing required argument(s)
nsh> ifconfig
eth0      Link encap:Ethernet HWaddr 00:e0:de:ad:be:ef at UP mtu 576
          inet addr:192.168.0.50 DRaddr:192.168.0.1 Mask:255.255.0
lo        Link encap:Local Loopback at RUNNING mtu 1518
          inet addr:127.0.0.1 DRaddr:127.0.0.1 Mask:255.0.0.0

nsh> ping 192.168.0.1
PING 192.168.0.1 56 bytes of data
56 bytes from 192.168.0.1: icmp_seq=0 time=10.0 ms
56 bytes from 192.168.0.1: icmp_seq=1 time=0.0 ms
56 bytes from 192.168.0.1: icmp_seq=2 time=0.0 ms
56 bytes from 192.168.0.1: icmp_seq=3 time=0.0 ms
56 bytes from 192.168.0.1: icmp_seq=4 time=0.0 ms
56 bytes from 192.168.0.1: icmp_seq=5 time=0.0 ms
56 bytes from 192.168.0.1: icmp_seq=6 time=0.0 ms
56 bytes from 192.168.0.1: icmp_seq=7 time=0.0 ms
56 bytes from 192.168.0.1: icmp_seq=8 time=0.0 ms
56 bytes from 192.168.0.1: icmp_seq=9 time=0.0 ms
10 packets transmitted, 10 received, 0% packet loss, time 10100 ms
rtt min/avg/max/mdev = 0.000/1.000/10.000/3.000 ms
nsh>
```

Con la configuración de Ethernet lista, únicamente debíamos realizar la copia del Example de `udp_client` que habíamos realizado posteriormente para la BeagleBone Black, sin embargo, al realizar la copia tuvimos cierto inconveniente ya que los Examples aparecen compilados para la Beagle, por lo que tuvimos que realizar un `distclean` y volver a realizar el `./tools/configure.sh` `raspberrypi-pico:enc28j60` y con esto ya aparecía el Example en el `menuconfig`

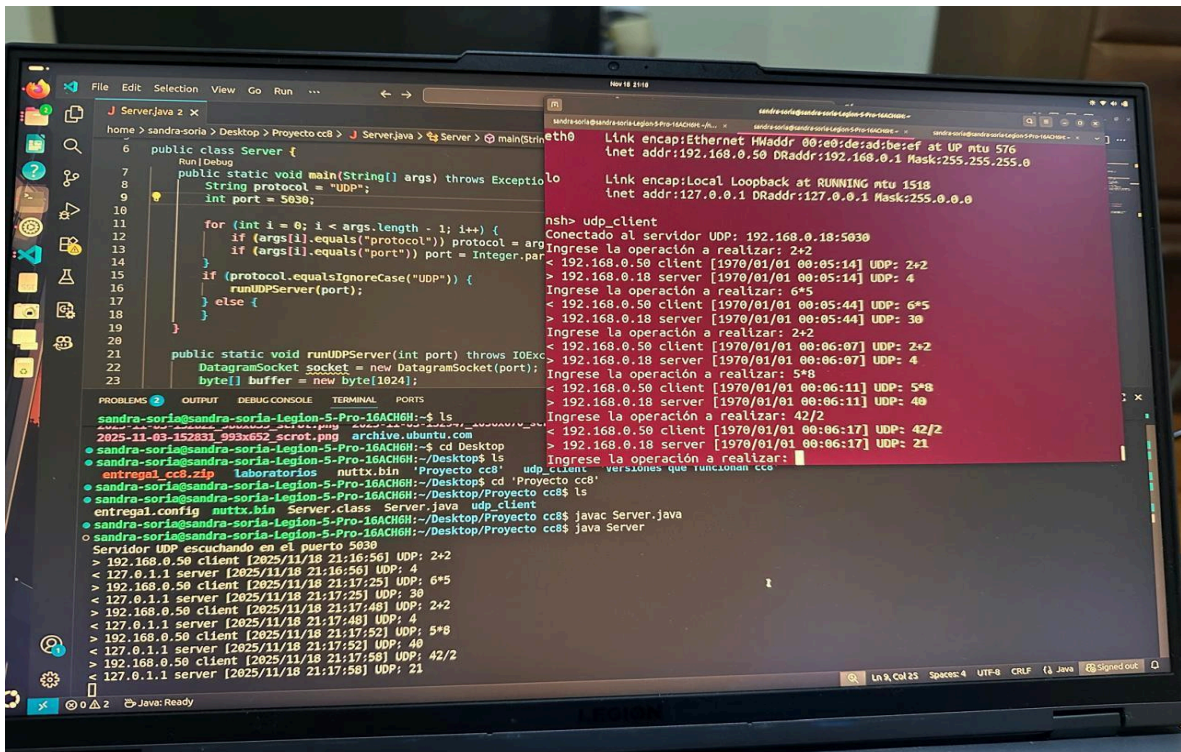


Al finalizar lo anterior fue necesario modificar el código colocando la dirección IP de la computadora como la dirección destino, la cual es la 192.168.0.18, teniendo esto ya finalizado se realizó la parte de la compilación del make y seguidamente se cargó el programa a la tarjeta para probar el laboratorio, el cual funcionó de manera exitosa.



```
EXPLORER  C udp_client_main.c
UDP_CLIENT
udp_client_main.c SERVER_IP
1 #include <stdio.h>
2 #include <stdlib.h>

sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eno1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 6c:24:08:73:bf:75 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
3: wlp4s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 30:03:c8:2c:cf:5d brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.18/24 brd 192.168.0.255 scope global dynamic noprefixroute wlp4s0
        valid_lft 1877sec preferred_lft 1877sec
    inet6 2803:d100:98d8:726e:b7d:34aa:c64c:acca/64 scope global temporary dynamic
        valid_lft 3600sec preferred_lft 3600sec
    inet6 2803:d100:98d8:726e:60ac:e9b7:9f93:b6d/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 3600sec preferred_lft 3600sec
    inet6 fe80::9f81:da6d:3c0:a16f/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
5: enx7272fe208d92: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 72:72:fe:20:8d:92 brd ff:ff:ff:ff:ff:ff
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H:~$
```



```
File Edit Selection View Go Run ...
home > sandra-soria > Desktop > Proyecto cc8 > Server.java > main$Server
6 public class Server {
7     public static void main(String[] args) throws Exception {
8         String protocol = "UDP";
9         int port = 5030;
10
11         for (int i = 0; i < args.length - 1; i++) {
12             if (args[i].equals("protocol")) protocol = args[i+1];
13             if (args[i].equals("port")) port = Integer.parseInt(args[i+1]);
14         }
15         if (protocol.equalsIgnoreCase("UDP")) {
16             runUDPServer(port);
17         } else {
18             // ...
19         }
20
21         public static void runUDPServer(int port) throws IOException {
22             DatagramSocket socket = new DatagramSocket(port);
23             byte[] buffer = new byte[1024];
24
25             while (true) {
26                 DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
27                 socket.receive(packet);
28                 // ...
29             }
30         }
31     }
32 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H:~$ ls
2025-11-03-152831_993kx62_screenshot.png archive.ubuntu.com
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H:~/Desktop$ cd Desktop
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H:~/Desktop$ ls
entregal_cc8.zip laboratorios nuttx.bin 'Proyecto cc8'
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H:~/Desktop$ cd 'Proyecto cc8'
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H:~/Desktop/Proyecto cc8$ ls
entregal.config nuttx.bin Server.class Server.java udp_client
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H:~/Desktop/Proyecto cc8$ javac Server.java
sandra-soria@sandra-soria-Legion-5-Pro-16ACH6H:~/Desktop/Proyecto cc8$ java Server

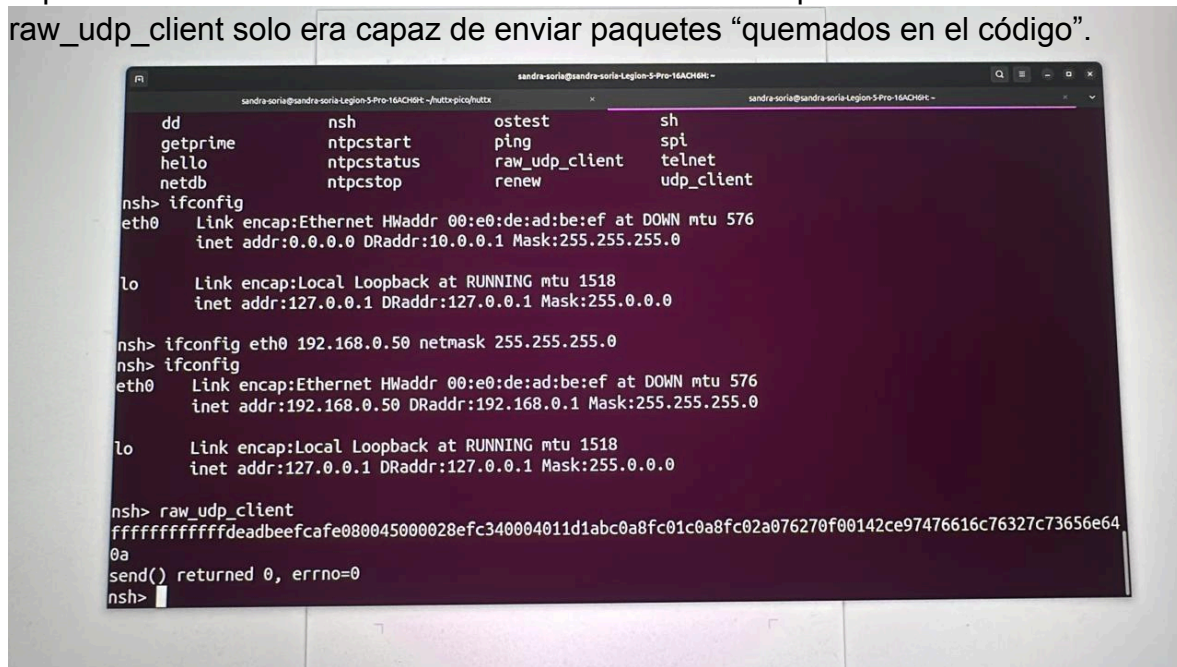
Servidor UDP escuchando en el puerto 5030
> 192.168.0.50 client [2025/11/18 21:16:56] UDP: 2+2
< 127.0.1.1 server [2025/11/18 21:16:56] UDP: 4
> 192.168.0.50 client [2025/11/18 21:17:25] UDP: 6*5
< 127.0.1.1 server [2025/11/18 21:17:25] UDP: 30
> 192.168.0.50 client [2025/11/18 21:17:48] UDP: 2+2
< 127.0.1.1 server [2025/11/18 21:17:48] UDP: 4
> 192.168.0.50 client [2025/11/18 21:17:52] UDP: 5*8
< 127.0.1.1 server [2025/11/18 21:17:52] UDP: 40
> 192.168.0.50 client [2025/11/18 21:17:58] UDP: 42/2
< 127.0.1.1 server [2025/11/18 21:17:58] UDP: 21

eth0 Link encap:Ethernet HWaddr 00:e0:de:ad:be:ef at UP mtu 576
      inet addr:192.168.0.50 Bcast:192.168.0.1 Mask:255.255.0
      Link encap:Local Loopback at RUNNING mtu 1518
      inet addr:127.0.0.1 Bcast:127.0.0.1 Mask:255.0.0.0

msh> udp_client
Conectado al servidor UDP: 192.168.0.18:5030
Ingrese la operación a realizar: 2+2
> 192.168.0.50 client [1970/01/01 00:05:14] UDP: 2+2
< 192.168.0.18 server [1970/01/01 00:05:14] UDP: 4
Ingrese la operación a realizar: 6*5
> 192.168.0.50 client [1970/01/01 00:05:44] UDP: 6*5
< 192.168.0.18 server [1970/01/01 00:05:44] UDP: 30
Ingrese la operación a realizar: 2+2
> 192.168.0.50 client [1970/01/01 00:06:07] UDP: 2+2
< 192.168.0.18 server [1970/01/01 00:06:07] UDP: 4
Ingrese la operación a realizar: 5*8
> 192.168.0.50 client [1970/01/01 00:06:11] UDP: 5*8
< 192.168.0.18 server [1970/01/01 00:06:11] UDP: 40
Ingrese la operación a realizar: 42/2
> 192.168.0.50 client [1970/01/01 00:06:17] UDP: 42/2
< 192.168.0.18 server [1970/01/01 00:06:17] UDP: 21
Ingrese la operación a realizar:
udp_client versiones que funcionan cc8
```


Errores con Raw Sockets dinámicos para Raspberry Pi Pico

Una vez finalizada la configuración de Ethernet, comenzamos con la implementación dinámica de los raw sockets para NuttX. Inicialmente raw_udp_client solo era capaz de enviar paquetes “quemados en el código”.



```
sandra-soria@sandra-soria-Legion-5-Pro-16ACH9t - /nuttx/pico/nuttx
dd      nsh      otest      sh
getprime ntpcstart ping      spi
hello   ntpcstatus raw_udp_client telnet
netdb   ntpcstop  renew      udp_client

nsh> ifconfig
eth0    Link encap:Ethernet HWaddr 00:e0:de:ad:be:ef at DOWN mtu 576
        inet addr:0.0.0.0 BDraddr:10.0.0.1 Mask:255.255.255.0

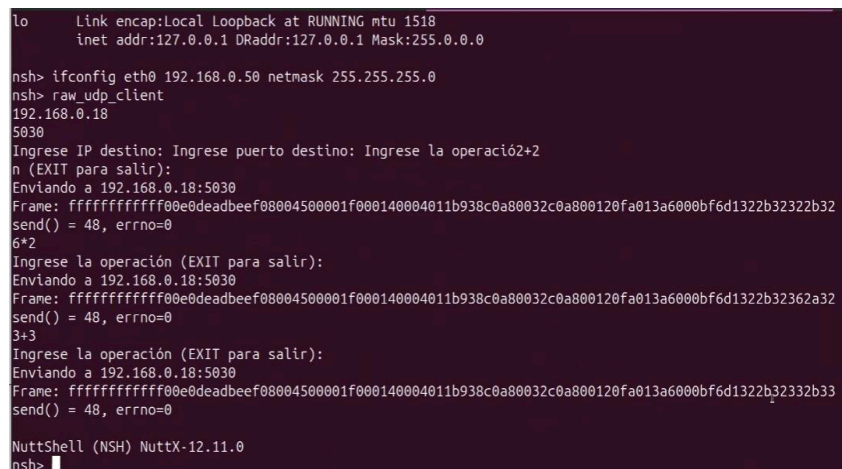
lo      Link encap:Local Loopback at RUNNING mtu 1518
        inet addr:127.0.0.1 BDraddr:127.0.0.1 Mask:255.0.0.0

nsh> ifconfig eth0 192.168.0.50 netmask 255.255.255.0
nsh> ifconfig
eth0    Link encap:Ethernet HWaddr 00:e0:de:ad:be:ef at DOWN mtu 576
        inet addr:192.168.0.50 BDraddr:192.168.0.1 Mask:255.255.255.0

lo      Link encap:Local Loopback at RUNNING mtu 1518
        inet addr:127.0.0.1 BDraddr:127.0.0.1 Mask:255.0.0.0

nsh> raw_udp_client
ffffffffffffdeadbeef00045000028efc340004011d1abc0a8fc01c0a8fc02a076270f00142ce97476616c76327c73656e64
0a
send() returned 0, errno=0
nsh>
```

Utilizando el código base comenzamos a modificarlo para que funcionara como el laboratorio 1 y 7. Sin embargo, al intentar implementarlo aparecieron ciertos errores. Uno de ellos estaba relacionado con el manejo de las operaciones, ya que se mantiene la operación anterior y luego se añadía la operación que el usuario escribía, por lo que estaba incorrecto y no llegaba nada al servidor. Para solucionar este problema lo que realizamos fue que en el arreglo de bytes se debía quitar el payload default.

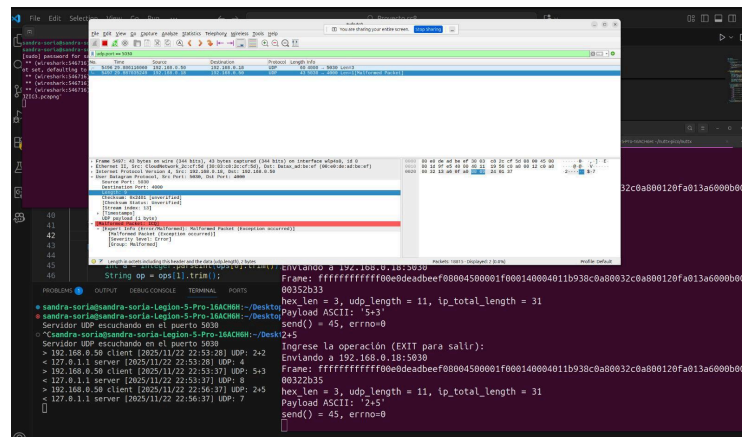


```
lo      Link encap:Local Loopback at RUNNING mtu 1518
        inet addr:127.0.0.1 BDraddr:127.0.0.1 Mask:255.0.0.0

nsh> ifconfig eth0 192.168.0.50 netmask 255.255.255.0
nsh> raw_udp_client
192.168.0.18
5030
Ingrese IP destino: Ingrese puerto destino: Ingrese la operación:2+2
n (EXIT para salir):
Enviando a 192.168.0.18:5030
Frame: ffffffff00deadbeef0004500001f000140004011b938c0a80032c0a800120fa013a6000bf6d1322b32322b32
send() = 48, errno=0
6*2
Ingrese la operación (EXIT para salir):
Enviando a 192.168.0.18:5030
Frame: ffffffff00deadbeef0004500001f000140004011b938c0a80032c0a800120fa013a6000bf6d1322b32322b32
send() = 48, errno=0
3+3
Ingrese la operación (EXIT para salir):
Enviando a 192.168.0.18:5030
Frame: ffffffff00deadbeef0004500001f000140004011b938c0a80032c0a800120fa013a6000bf6d1322b32322b33
send() = 48, errno=0

NuttShell (NSH) NuttX-12.11.0
nsh>
```


Una vez terminada la parte de envío, nos centramos en la recepción de paquetes desde el servidor hacia nuestro cliente. Para ello utilizamos Wireshark para observar el envío de paquetes por parte del servidor. En la captura se observa el mensaje “Malformed Packet” en un paquete UDP cuyo puerto destino es 4000. Este mensaje ocurre por como Wireshark interpreta ese puerto, ya que lo asocia con el protocolo de mensajería ICQ, por lo que decidimos cambiar el puerto utilizado en nuestro código.



Además, nos dimos cuenta de que en el código teníamos esto:

```
n = recvfrom(sockfd, rxbuf, sizeof(rxbuf), 0,
             (struct sockaddr*)&addr,
             (socklen_t*)&sizeof(addr));
```

Lo cual está incorrecto porque el parámetro `recvfrom` no está esperando un número, sino un puntero a una variable donde va a guardar el tamaño de la dirección, entonces se obtenían frames de tipo broadcast de UDP. Por lo que para solucionarlo realizamos lo siguiente:

```
socklen_t addrlen = sizeof(addr);
n = recvfrom(sockfd, rxbuf, sizeof(rxbuf), 0,
             (struct sockaddr*)&addr,
             &addrlen);
```

Al crear una variable `socklen_t addrlen = sizeof(addr);` y utilizar `&addrlen` en la llamada a `recvfrom`, la recepción de paquetes funcionaba correctamente, ya que la función necesita la dirección de una variable donde guardar el tamaño de la dirección.


```
close(sockfd);
Atlas
defconfig
help
mv
steep
watch

sandra.soria@sandra.soria-Legion-5-Pro-16ACH6H: ~/nuttx-pico/nuttx
sandra.soria@sandra.soria-Legion-5-Pro-16ACH6H: ~/nuttx-bbb-project/nuttx
sandra.soria@sandra.soria-Legion-5-Pro-16ACH6H: ~/nuttx-pico/nuttx

.config - NuttX/x86_64 Configuration
-> Networking Support -> Driver buffer configuration

Driver buffer configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes
features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*]
built-in [ ] excluded <M> module < > module capable

(1514) Ethernet packet buffer size
(2) Driver I/O guard size
(14) Data Link Layer(L2) Guard size of Network buffer(IOB)
(0) Net Default Receive buffer size

<Select> < Exit > < Help > < Save > < Load >
```